

Python for Data Science

Buổi 3: Nhập Môn Pandas – Series, DataFrame và Indexing

PGS-TS Nguyễn Mạnh Hùng

August 18, 2026

Cấu trúc Buổi 3

- 1 Ôn tập Buổi 2 và định vị Pandas trong Data Science Workflow
- 2 Module 9: `pandas.Series`
- 3 Module 10: `pandas.DataFrame`
- 4 Module 11: Đọc/Ghi dữ liệu với Pandas (I/O)
- 5 Module 12: Indexing, Selection và các phép toán trên DataFrame
- 6 Bài tập áp dụng trên dữ liệu thực (Handbook & Python for Data Analysis)

Lưu ý phạm vi

Buổi học tập trung nền tảng Pandas: Series, DataFrame, đọc/ghi dữ liệu, indexing và selection. Missing data, hierarchical indexing, merge/join, groupby và pivot table sẽ học ở Buổi 4–5.

Nhắc lại Buổi 2

- ndarray: mảng đồng nhất kiểu dữ liệu, tính toán vectorized, broadcasting.
- structured array: mỗi "cột" có tên và kiểu dữ liệu riêng – ý tưởng nền tảng của bảng dữ liệu.
- Module 8: dùng NumPy thuần để huấn luyện hồi quy tuyến tính (Gradient Descent, SVD).

Câu hỏi mở đầu

NumPy xử lý số liệu rất tốt, nhưng dữ liệu thực tế (bảng Excel, file CSV, log giao dịch...) có tên cột, chỉ mục có ý nghĩa (ngày tháng, mã khách hàng) và nhiều kiểu dữ liệu trộn lẫn. Làm sao xử lý hiệu quả?

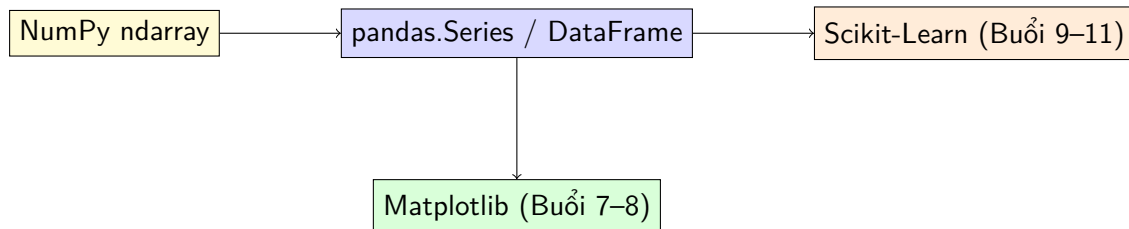
Vì sao NumPy chưa đủ cho dữ liệu thực tế?

	NumPy ndarray	Nhu cầu thực tế
Nhãn cột	Không có, chỉ có vị trí	"age", "monthly_fee", "churn"
Chỉ mục (index)	Vị trí số nguyên 0,1,2,...	Ngày tháng, mã khách hàng
Kiểu dữ liệu	Đồng nhất toàn mảng	Số, chuỗi, ngày tháng trộn lẫn
Dữ liệu thiếu	Không hỗ trợ trực tiếp	Rất phổ biến (NaN, null)

Pandas

Pandas xây dựng trên nền ndarray, bổ sung **nhãn** (label) cho hàng và cột, cho phép trộn kiểu dữ liệu và xử lý dữ liệu thiếu – giải quyết trực tiếp các hạn chế trên (McKinney, 2010).

Pandas trong hệ sinh thái Data Science



Series và DataFrame chính là ndarray được "gắn nhãn" – mọi kỹ thuật vectorization, broadcasting, boolean masking đã học ở Buổi 2 tiếp tục áp dụng trực tiếp trong Pandas.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Tạo Series từ list, dict và ndarray.
- 2 Truy cập phần tử Series bằng nhãn (label) và vị trí (position).
- 3 Giải thích cơ chế index alignment khi thực hiện phép toán giữa hai Series.

Series là gì?

Khái niệm

Series là một mảng một chiều có **nhãn** (index) đi kèm – có thể xem là ndarray + index.

```
import pandas as pd

scores = pd.Series([90, 85, 78, 92], index=["An", "Binh", "Chi", "Dung"])
print(scores)
# An      90
# Binh    85
# Chi     78
# Dung    92
# dtype: int64
```

Tạo Series bằng nhiều cách

```
import numpy as np

s1 = pd.Series([1, 2, 3])                # index mặc định: 0, 1, 2
s2 = pd.Series(np.array([1.5, 2.5, 3.5])) # từ ndarray

population = {"HCM": 9.0, "HN": 8.4, "DN": 1.2} # từ dict
s3 = pd.Series(population)                    # key -> index

s4 = pd.Series(5, index=["a", "b", "c"])      # giá trị lặp lại
```

Khi tạo từ dict, các *key* tự động trở thành *index* – đây là điểm khác biệt quan trọng so với ndarray thuần.

Truy cập phần tử: Label vs Position

```
scores = pd.Series([90, 85, 78, 92], index=["An", "Binh", "Chi", "Dung"])

print(scores["An"])          # 90 - truy cap theo nhan (label)
print(scores.iloc[0])        # 90 - truy cap theo vi tri (position)

print(scores[["An", "Chi"]])  # nhieu nhan cung luc
print(scores.iloc[0:2])      # slicing theo vi tri
```

Lưu ý

Truy cập trực tiếp `scores[0]` có thể gây nhầm lẫn nếu index không phải số nguyên liên tục – nên dùng tường minh `.loc[]` (theo nhãn) hoặc `.iloc[]` (theo vị trí), sẽ trình bày kỹ ở Module 12.

Index Alignment: phép toán giữa hai Series

```
s1 = pd.Series([90, 85, 78], index=["An", "Binh", "Chi"])
s2 = pd.Series([1, 2, 3], index=["An", "Binh", "Emy"])

print(s1 + s2)
# An      91.0
# Binh    87.0
# Chi      NaN    <- Chi không có trong s2
# Emy      NaN    <- Emy không có trong s1
```

Pandas tự động **căn chỉnh theo index** (index alignment) trước khi tính toán: nhãn nào không xuất hiện ở cả hai Series sẽ cho kết quả NaN – khác hoàn toàn với broadcasting theo vị trí của NumPy.

Tổng kết Module 9

- `Series` = `ndarray` có index gắn nhãn.
- Tạo từ `list`, `dict` (`key` $\rightarrow$ `index`), `ndarray`, hoặc giá trị lặp lại.
- Truy cập: nhãn (`s["ten"]`, `.loc`) hoặc vị trí (`.iloc`).
- Phép toán giữa hai `Series` tự động căn chỉnh theo index (index alignment).

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- ➊ Tạo DataFrame từ dict, list of dict, và ndarray.
- ➋ Đọc nhanh cấu trúc và thống kê mô tả của một DataFrame.
- ➌ Truy cập, thêm và xóa cột trong DataFrame.

DataFrame là gì?

Khái niệm

DataFrame là bảng dữ liệu hai chiều: tập hợp nhiều Series **chia sẻ chung một index**, mỗi Series là một cột.

index

Series "name"

Series "age"

Series "fee"

Đây chính xác là ý tưởng structured array đã học ở Buổi 2 – DataFrame là phiên bản đầy đủ tính năng hơn, hỗ trợ index linh hoạt và thao tác dữ liệu phong phú.

Tạo DataFrame từ dict

```
data = {
    "name": ["An", "Binh", "Chi"],
    "age": [25, 32, 41],
    "monthly_fee": [12.5, 35.0, 18.0]
}
df = pd.DataFrame(data)
print(df)
```

#	name	age	monthly_fee
# 0	An	25	12.5
# 1	Binh	32	35.0
# 2	Chi	41	18.0

Mỗi *key* của dict trở thành một cột; index mặc định là số nguyên 0, 1, 2, ...

Tạo DataFrame: các cách khác

```
records = [  
    {"name": "An", "age": 25},  
    {"name": "Binh", "age": 32},  
]  
df2 = pd.DataFrame(records)           # tu list of dict  
  
arr = np.random.randint(0, 100, (3, 2))  
df3 = pd.DataFrame(arr, columns=["x", "y"])    # tu ndarray  
  
df4 = pd.DataFrame(data, index=["c1", "c2", "c3"])  # tu dinh index tuy chinh
```

Thuộc tính của DataFrame

```
print(df.index)           # RangeIndex(start=0, stop=3, step=1)
print(df.columns)         # Index(['name', 'age', 'monthly_fee'])
print(df.values)          # ndarray 2 chieu ben duoi
print(df.shape)           # (3, 3)
print(df.dtypes)          # kieu du lieu tung cot
```

dtypes thường là int64, float64 cho số và object (chuỗi ký tự, tùy phiên bản pandas có thể là string) cho văn bản – so sánh với dtype của từng trường trong structured array ở Buổi 2.

Xem nhanh dữ liệu

```
df.head(2)           # 2 dòng đầu  
df.tail(2)           # 2 dòng cuối  
df.info()             # tổng quan: số dòng, kiểu dữ liệu, bộ nhớ  
df.describe()         # thống kê mô tả: mean, std, min, max, quartile
```

Thói quen tốt

Trước khi phân tích bất kỳ tập dữ liệu nào, luôn bắt đầu bằng `head()`, `info()` và `describe()` để nắm cấu trúc, kiểu dữ liệu và phát hiện bất thường sơ bộ.

Truy cập, thêm và xóa cột

```
print(df["age"])          # truy cap 1 cot -> tra ve Series
print(df.age)             # tuong duong (neu ten cot hop le nhu ten bien)
print(df[["name", "age"]]) # nhieu cot -> tra ve DataFrame

df["is_adult"] = df["age"] >= 18    # them cot moi (vectorized)
df["age_next_year"] = df["age"] + 1

df = df.drop(columns=["is_adult"])  # xoa cot
```

Tổng kết Module 10

- DataFrame = nhiều Series chia sẻ chung index – mỗi cột là một Series.
- Tạo từ dict, list of dict, ndarray.
- head/tail/info/describe: bước đầu tiên khi khám phá dữ liệu mới.
- Truy cập cột bằng `df["col"]`; thêm cột mới bằng phép gán vectorized.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Đọc dữ liệu từ file CSV với các tham số phổ biến.
- 2 Đọc/ghi các định dạng dữ liệu khác (Excel, JSON).
- 3 Áp dụng để nạp một tập dữ liệu thực tế vào DataFrame.

Đọc dữ liệu với read_csv

```
df = pd.read_csv("customers.csv")
```

```
df = pd.read_csv(  
    "customers.csv",  
    sep="," ,          # ký tự phân cách  
    header=0,          # dòng nào là header  
    index_col="customer_id", # cột nào làm index  
    usecols=["customer_id", "age", "monthly_fee"], # chỉ đọc các cột cần  
    na_values=["NA", "?", ""], # các giá trị coi là thiếu  
    dtype={"age": "int32"}      # ép kiểu dữ liệu  
)
```

read_csv có hơn 50 tham số tùy chỉnh – trong thực hành chỉ cần nắm vững nhóm tham số về phân tách, header, index và missing value.

Đọc/Ghi các định dạng khác

```
df_excel = pd.read_excel("report.xlsx", sheet_name="Sheet1")
df_json = pd.read_json("data.json")

df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
df.to_json("output.json")
```

Quy ước đối xứng `read_*`() / `to_*`() giúp Pandas làm việc thống nhất với nhiều nguồn dữ liệu khác nhau mà không cần đổi tư duy xử lý.

Thực hành: nạp dữ liệu thực tế

```
# Ví dụ minh họa cấu trúc dữ liệu US Baby Names
# (Python for Data Analysis, Module 4 - Real-World Applications)
names = pd.read_csv("yob2010.txt", names=["name", "sex", "births"])

print(names.head())
print(names.shape)
print(names["sex"].unique())          # các giá trị duy nhất trong cột sex
print(names.sort_values("births", ascending=False).head(10))
```

Case study *US Baby Names* (PDA, Module 4) là ví dụ điển hình để luyện đọc CSV, kiểm tra cấu trúc và sắp xếp dữ liệu thực tế trước khi đi sâu vào các phép biến đổi phức tạp hơn ở Buổi 4–5.

Tổng kết Module 11

- `read_csv`: tham số quan trọng – `sep`, `header`, `index_col`, `na_values`, `dtype`.
- `read_excel`, `read_json` và các hàm `to_*`() tương ứng để ghi dữ liệu.
- Luôn kiểm tra `shape`, `head()`, `dtypes` ngay sau khi đọc dữ liệu mới.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân biệt và sử dụng đúng `.loc[]` và `.iloc[]`.
- 2 Lọc dữ liệu bằng boolean indexing và `query()`.
- 3 Áp dụng phép toán vectorized và `apply()` trên DataFrame.
- 4 Sắp xếp DataFrame theo giá trị hoặc theo index.

.loc[] và .iloc[]

```
df = pd.DataFrame(data, index=["c1", "c2", "c3"])

print(df.loc["c1"])           # 1 dòng, theo nhãn
print(df.loc["c1", "age"])     # 1 giá trị, theo nhãn (dòng, cột)
print(df.loc[:, ["name", "age"]]) # tất cả dòng, một số cột

print(df.iloc[0])             # 1 dòng, theo vị trí
print(df.iloc[0, 1])          # 1 giá trị, theo vị trí (dòng, cột)
print(df.iloc[0:2, 0:2])      # slicing theo vị trí
```

Ghi nhớ

.loc[]: chọn theo **nhãn** (label) – cần trên được bao gồm khi slicing. .iloc[]: chọn theo **vị trí** (position) – giống quy ước slicing của NumPy/Python.

Boolean Indexing trên DataFrame

```
print(df[df["age"] > 25])                # loc theo 1 dieu kien
print(df[(df["age"] > 25) & (df["monthly_fee"] > 15)])    # ket hop AND
print(df[(df["age"] < 25) | (df["monthly_fee"] > 30)])    # ket hop OR

print(df.query("age > 25 and monthly_fee > 15"))        # cu phap query()
```

Cùng nguyên lý boolean mask đã học với NumPy ở Buổi 2 (Module 7): & và | thay cho and/or, luôn đặt từng điều kiện trong ngoặc đơn.

Phép toán Vectorized và apply()

```
df["fee_with_tax"] = df["monthly_fee"] * 1.1          # vectorized, ke thua NumPy

def classify_age(age):
    return "senior" if age > 30 else "junior"

df["age_group"] = df["age"].apply(classify_age)         # ap dung ham cho tung phan tu
df["full_info"] = df.apply(
    lambda row: f"{row['name']} ({row['age']})", axis=1 # axis=1: theo tung dong
)
```

Ưu tiên phép toán vectorized (nhANH) khi có thể; chỉ dùng apply() khi logic quá phức tạp để viết vectorized trực tiếp.

Sorting DataFrame

```
print(df.sort_values("age"))  
print(df.sort_values("age", ascending=False))  
print(df.sort_values(["age", "monthly_fee"]))  
  
print(df.sort_index())
```

tang dan theo 1 cot
giam dan
nhieu cot

sap xep theo index

Tổng kết Module 12

- `.loc[]` theo nhãn, `.iloc[]` theo vị trí – không trộn lẫn hai cú pháp.
- Boolean indexing: `df[condition]`, kết hợp `&/|`; hoặc dùng `query()`.
- Ưu tiên phép toán vectorized; `apply()` khi cần logic tùy biến.
- `sort_values()` theo cột, `sort_index()` theo index.

Bảng tổng hợp các thao tác Pandas quan trọng

Nhóm	Hàm/thao tác	Mục đích
Tạo dữ liệu	<code>pd.Series</code> , <code>pd.DataFrame</code>	Khởi tạo Series/DataFrame
Đọc/Ghi	<code>read_csv</code> , <code>read_excel</code> , <code>to_csv</code>	I/O với file dữ liệu
Khám phá	<code>head</code> , <code>tail</code> , <code>info</code> , <code>describe</code> , <code>shape</code>	Hiểu cấu trúc dữ liệu
Truy cập	<code>df["col"]</code> , <code>.loc[]</code> , <code>.iloc[]</code>	Chọn dòng/cột
Lọc dữ liệu	<code>df[condition]</code> , <code>query()</code>	Boolean indexing
Biến đổi	phép toán vectorized, <code>apply()</code>	Tạo cột mới, biến đổi dữ liệu
Sắp xếp	<code>sort_values</code> , <code>sort_index</code>	Sắp xếp theo giá trị/index

Khởi động: bài tập với dữ liệu tự sinh

- 1 Tạo một DataFrame từ dict mô tả 10 khách hàng (tên, tuổi, phí hàng tháng, số ticket hỗ trợ); dùng `describe()` để thống kê tổng quan.
- 2 Thêm cột phân loại khách hàng (`apply()` hoặc `vectorized`) dựa trên điều kiện về tuổi và phí hàng tháng, sau đó sắp xếp theo cột mới.

Gợi ý mở rộng

Thử so sánh thời gian chạy giữa cách viết `vectorized` và `apply()` bằng `%timeit` (đã học ở Buổi 2) trên một DataFrame vài trăm nghìn dòng.

Bộ dữ liệu thực hành

Các bài tập dưới đây dùng dữ liệu **thật**, lấy trực tiếp từ hai kho tham khảo của môn học.

Dataset	Nguồn	Đường dẫn (trong repo)	Cột dữ liệu
US President Heights	Handbook	notebooks/data/president_heights.csv	order, name, height(cm)
US Baby Names (theo năm)	PDA	datasets/babynames/yobYYYY.txt	name, sex, births (không header)
Bitly / USA.gov (JSON Lines)	PDA	datasets/bitly_usagov/example.txt	mỗi dòng 1 JSON: tz, cy, c, ...
US State Population	Handbook	notebooks/data/state-population.csv	state/region, ages, year, population

Handbook = *jakevdp/PythonDataScienceHandbook* (nhánh master); PDA = *datacamp/python-for-data-analysis* (nhánh main).

Cách nạp dữ liệu trực tiếp từ GitHub

```
import pandas as pd

url = ("https://raw.githubusercontent.com/jakevdp/"
      "PythonDataScienceHandbook/master/notebooks/data/president_heights.csv")
heights = pd.read_csv(url)      # đọc trực tiếp từ URL, không cần tải file về máy
```

Hai cách lấy dữ liệu

- 1 `pd.read_csv(url)` với URL dạng `raw.githubusercontent.com` – nhanh, không cần clone repo.
- 2 Clone repo về máy (`git clone <url>`), sau đó đọc theo đường dẫn cục bộ – phù hợp khi làm việc với file lớn hoặc offline.

Bài tập 1 (Handbook): US President Heights

Dữ liệu: `notebooks/data/president_heights.csv` (45 dòng, cột `order`, `name`, `height(cm)`).

- 1 Đọc dữ liệu bằng `pd.read_csv()`; dùng `describe()` để xem chiều cao trung bình, min, max.
- 2 Dùng `.loc/.iloc` để lấy ra tổng thống thứ 16 (Abraham Lincoln) theo hai cách khác nhau.
- 3 Dùng boolean indexing: liệt kê các tổng thống cao hơn 188cm.
- 4 Dùng `sort_values()` để tìm ra 3 tổng thống cao nhất và 3 tổng thống thấp nhất.

Bài tập 2 (Python for Data Analysis): US Baby Names

Dữ liệu: `datasets/babynames/yob2010.txt` (không có header, cột name, sex, births).

- 1 Đọc dữ liệu bằng `pd.read_csv(path, names=["name", "sex", "births"])`.
- 2 Dùng boolean indexing tách riêng tên bé trai (`sex == "M"`) và bé gái (`sex == "F"`).
- 3 Dùng `sort_values("births", ascending=False)` để tìm top 10 tên phổ biến nhất mỗi giới tính.
- 4 So sánh với file `yob1880.txt`: tên phổ biến nhất năm 1880 có còn phổ biến vào năm 2010 không?

Mở rộng

Thử lặp qua nhiều file `yobYYYY.txt` và ghép lại bằng `pd.concat()` – kỹ thuật này sẽ học chính thức ở Buổi 4.

Bài tập 3 (nâng cao): Bitly / USA.gov – dữ liệu JSON Lines

Dữ liệu: `datasets/bitly_usagov/example.txt` – mỗi dòng là một object JSON (không phải CSV).

```
import json

path = "datasets/bitly_usagov/example.txt"
with open(path) as f:
    records = [json.loads(line) for line in f]    # moi dong -> 1 dict

df = pd.DataFrame(records)    # list of dict -> DataFrame (nhu da hoc Module 10)
```

- 1 Kiểm tra `df.shape`, `df.columns`, `df.dtypes` – có bao nhiêu bản ghi, cột nào có dữ liệu thiếu?
- 2 Dùng boolean indexing lọc ra các bản ghi có `tz` (timezone) khác rỗng.
- 3 Lấy ra 10 dòng có `cy` (city) trùng với một thành phố cụ thể, ví dụ "Danvers".

Bài tập 4 (mở rộng – chuẩn bị Buổi 4): US State Population

Dữ liệu: `notebooks/data/state-population.csv` (state/region, ages, year, population).

- 1 Đọc dữ liệu; dùng boolean indexing lọc riêng các dòng `ages == "total"`.
- 2 Với bang "CA", sắp xếp theo year và quan sát dân số thay đổi qua thời gian.
- 3 Tìm năm có tổng dân số (`ages == "total"`) cao nhất trong toàn bộ dữ liệu bằng `sort_values()`.

Vì sao đây là bài tập "mở rộng"?

Cột `state/region` ở đây là mã viết tắt ("CA", "AL"...), trong khi tên đầy đủ của bang nằm ở một file khác (`state-abbrevs.csv`). Việc **kết hợp hai bảng** này chính là kỹ thuật merge sẽ học chính thức ở Buổi 4.

Tài liệu tham khảo I

- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- The pandas development team. (2020). *pandas-dev/pandas: Pandas (Version 1.0)* [Software]. Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – dữ liệu president_heights.csv, state-population.csv]. GitHub. <https://github.com/jakevdp/PythonDataScienceHandbook>
- Pugh, D. R. (n.d.). *python-for-data-analysis* [Kho mã nguồn – dữ liệu babynames, bitly_usagov], fork của McKinney, W., *pydata-book*. GitHub. <https://github.com/davidrpugh/python-for-data-analysis>

Chuẩn bị cho buổi tiếp theo: Data Loading, Storage, and File Formats

Thông điệp

Series và DataFrame là ndarray được gắn nhãn: mọi kỹ thuật vectorization, broadcasting và boolean masking của NumPy tiếp tục là nền tảng khi làm việc với Pandas. Buổi 4 sẽ mở rộng `read_csv()` đã học sang nhiều định dạng và nguồn dữ liệu khác nhau, trước khi Buổi 5 đi vào các phương pháp xử lý dữ liệu (missing data, kết hợp bảng).